The Coding Interview Patterns Playbook

Taras Goriachko

Course Introduction

Requirements

- Basic knowledge of any programming language - ideally Kotlin
- Optional: Understanding of the Gradle build system*
- **Preinstalled on computer:**
  - IntelliJ IDEA (Community Edition)**
  - Java 17***

---

*http://udemy.com/course/mastering-gradle/

*https://www.jetbrains.com/idea/download/?section=mac

**https://adoptium.net/en-GB/temurin/releases/?version=17&os=mac&arch=aarch64&package=jre

Welcome to the Course

- Why "Patterns" instead of "Problems"?
- Moving from memorization to intuition
- The 80/20 Rule: 19 patterns solve 80% of interviews

Welcome to the Course

- Why "Patterns" instead of "Problems"? 🧠
- Moving from memorization to intuition
- The 80/20 Rule: 19 patterns solve 80% of interviews

Welcome to the Course

- Why "Patterns" instead of "Problems"? 🧠
- Moving from memorization to intuition 💡
- The 80/20 Rule: 19 patterns solve 80% of interviews

Welcome to the Course

- Why "Patterns" instead of "Problems"? 🧠
- Moving from memorization to intuition 💡
- The 80/20 Rule: 19 patterns solve 80% of interviews 📈

How to Use This Course Effectively

- **Course Approach:** We will start by showing the brute-force solution, and then transition to the optimal solution.
- **In an Interview (Ideal Case):**
  1. **Identify:** Recognize pattern triggers in the problem text
  2. **Trace:** Dry-run the logic on a small input
  3. **Implement:** Code the solution without looking at hints

Interview Mindset: Thinking in Patterns

- Stop looking at the "Story" (e.g., "Alice has 3 apples...")
- Start looking at the **Data Flow**

Problem Trait	**Pattern Strategy**
Finding a sub-segment in a list	Sliding Window
Searching a sorted range	Binary Search
Shortest path in unweighted graph	BFS

Common Interview Mistakes

- **Mistake 1:** Starting to code before the logic is 100% clear
- **Mistake 2:** When stuck: avoiding brute force and trying to guess optimal solution
- **Mistake 3:** Forgetting to state Big O complexity
- **Mistake 4:** Not testing for $n = 0$ or $n = 1$
- **Mistake 5:** Not listening to the interviewer

Correction: Think out loud. Your interviewer is your collaborator, not your judge.

Common Interview Mistakes

- **Mistake 1:** Starting to code before the logic is 100% clear ❌
- **Mistake 2:** When stuck: avoiding brute force and trying to guess optimal solution
- **Mistake 3:** Forgetting to state Big O complexity
- **Mistake 4:** Not testing for $n = 0$ or $n = 1$
- **Mistake 5:** Not listening to the interviewer

Correction: Think out loud. Your interviewer is your collaborator, not your judge.

Common Interview Mistakes

- **Mistake 1:** Starting to code before the logic is 100% clear ❌
- **Mistake 2:** When stuck: avoiding brute force and trying to guess optimal solution ❌
- **Mistake 3:** Forgetting to state Big O complexity
- **Mistake 4:** Not testing for $n = 0$ or $n = 1$
- **Mistake 5:** Not listening to the interviewer

Correction: Think out loud. Your interviewer is your collaborator, not your judge.

Common Interview Mistakes

- **Mistake 1:** Starting to code before the logic is 100% clear ❌
- **Mistake 2:** When stuck: avoiding brute force and trying to guess optimal solution ❌
- **Mistake 3:** Forgetting to state Big O complexity ❌
- **Mistake 4:** Not testing for $n = 0$ or $n = 1$
- **Mistake 5:** Not listening to the interviewer

Correction: Think out loud. Your interviewer is your collaborator, not your judge.

Common Interview Mistakes

- **Mistake 1:** Starting to code before the logic is 100% clear ✗
- **Mistake 2:** When stuck: avoiding brute force and trying to guess optimal solution ✗
- **Mistake 3:** Forgetting to state Big O complexity ✗
- **Mistake 4:** Not testing for $n = 0$ or $n = 1$ ✗
- **Mistake 5:** Not listening to the interviewer

Correction: Think out loud. Your interviewer is your collaborator, not your judge.

Common Interview Mistakes

- **Mistake 1:** Starting to code before the logic is 100% clear ✗
- **Mistake 2:** When stuck: avoiding brute force and trying to guess optimal solution ✗
- **Mistake 3:** Forgetting to state Big O complexity ✗
- **Mistake 4:** Not testing for $n = 0$ or $n = 1$ ✗
- **Mistake 5:** Not listening to the interviewer ✗

Correction: Think out loud. Your interviewer is your collaborator, not your judge.

Course Structure

- Pattern 1: Two Pointers Pattern
- Pattern 2: Sliding Window
- Pattern 3: Monotonic Stack
- Pattern 4: Prefix Sum & Cumulative
- Pattern 5: Cyclic Sort
- Pattern 6: Merge Intervals
- Pattern 7: Modified Binary Search
- Pattern 8: Fast & Slow Pointers
- Pattern 9: Tree Depth First Search (DFS)
- Pattern 10: Tree Breadth First Search (BFS)
- Pattern 11: Matrix Traversal (Islands)
- Pattern 12: Union-Find (Disjoint Set)
- Pattern 13: Top 'K' Elements (Heap)
- Pattern 14: Dynamic Programming
- Pattern 15: Backtracking & DFS
- Pattern 16: Greedy Algorithms
- Pattern 17: Data Streams & Online
- Pattern 18: Advanced Patterns
- Pattern 19: Miscellaneous Techniques

Fundamentals & Big O

The Absolute Essentials

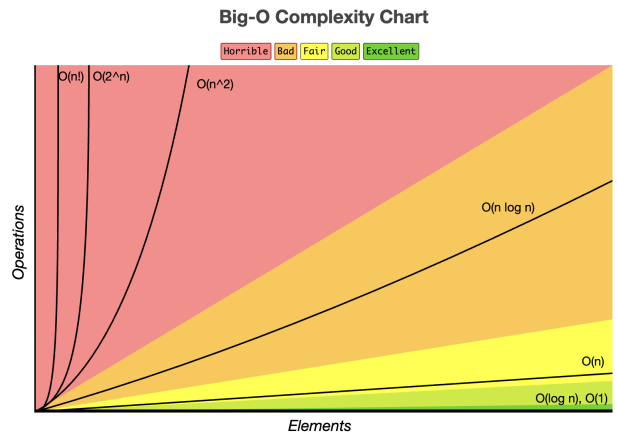
Before mastering coding patterns, you must master the fundamental building blocks.

- **Why?** Interviewers evaluate your ability to select the right tool for the job.
- **Key Areas:**
  - Analyzing efficiency (Big O Complexity).
  - Choosing the right data structures.
  - Writing idiomatic, clean code under pressure.

Big O Notation Demystified

How does the runtime or space requirement grow as input size N increases?

- **Time Complexity:** Count dominant operations.
  - $O(1)$ - Constant time.
  - $O(\log N)$ - Logarithmic (halving search space).
  - $O(N)$ - Linear (single pass).
  - $O(N \log N)$ - Linearithmic (sorting).
  - $O(N^2)$ - Quadratic (nested loops).
- **Space Complexity:** Measure auxiliary memory (call stack, maps, arrays).



bigocheatsheet.com

Growth Rates in Action ($N = 1,000,000$)

What do these complexities mean in practice for 1 million input elements?

Complexity	**Operations ($N = 10^6$)**	**Approx. Time (1GHz CPU)**
$O(1)$	1	$\approx$ 1 ns (Instant)
$O(\log N)$	≈ 20	$\approx$ 20 ns (Instant)
$O(N)$	$1,000,000$	$\approx$ 1 ms (Instant)
$O(N \log N)$	$\approx 2 \times 10^7$	$\approx$ 20 ms (Instant)
$O(N^2)$	10^{12}	$\approx$ 16.7 minutes (**Timeout**)
$O(2^N)$	$2^{1,000,000}$	$\gg$ Age of the Universe (**Impossible**)

Data Structures Crash Course

Data Structure	Strengths	Common Use Case
Array / List	Fast index access ($O(1)$)	Ordered traversal, sorting
HashMap / Map	Key lookup ($O(1)$ average)	Frequency counting, cache
HashSet / Set	Duplicate removal, search ($O(1)$)	Visited trackers, unique items
Queue / Deque	FIFO ordering ($O(1)$ push/pop)	BFS, level order traversal

Top Kotlin Idioms at a Glance

Idiom / Construct	Primary Use Case	Codebase Example
`intArrayOf` / `arrayOf`	Matrix/test declarations	`intArrayOf(2, 7, 11, 15)`
`until` operator	Forward loop (0 to *size* − 1)	`for (i in 0 until nums.size)`
Safe Call (`?.`)	Null-safe node traversal	`cur1 = cur1?.next`
`kotlin.math` (min/max)	Range clamping	`kotlin.math.min(price, minPrice)`
`mutableListOf` / `listOf`	Building outputs/collections	`val res = mutableListOf<List<Int>>()`
`getOrDefault`	Map access with fallback	`adj.getOrDefault(v, hashMapOf())`
`ArrayDeque`	BFS queue / Stack	`val deque = ArrayDeque<IntArray>()`
`return if` (Expression)	Conditional returns	`return if (val1 >= 0) val1 else val2`

Pattern 1: Two Pointers Pattern

What is the Two Pointers Pattern?

- **The Core Idea:** Use two indices (pointers) to iterate through a data structure simultaneously.
- **Why it works:** It reduces $O(n^2)$ nested loops to a single $O(n)$ pass.

Pattern Mechanics: Opposite Ends

When searching for a target in a **Sorted** array:

1. **Left Pointer:** Starts at index 0.
2. **Right Pointer:** Starts at index $n - 1$.
3. **Decision:**
   - If $sum > target$: Move Right pointer left ($\leftarrow$) to decrease sum.
   - If $sum < target$: Move Left pointer right ($\rightarrow$) to increase sum.

Problem: Zero-Sum Triplets

Scenario: Given an integer array, find all unique triplets $[a, b, c]$ such that their sum equals exactly zero. The solution set must not contain duplicate triplets.

Examples:

- **Input:** `[-1, 0, 1, 2, -1, -4]` $\rightarrow$ **Output:** `[[-1, -1, 2], [-1, 0, 1]]`
- **Input:** `[0, 1, 1]` $\rightarrow$ **Output:** `[]`

Problem: Maximizing Container Area

Scenario: Given an array representing wall heights, find two walls that, together with the horizontal axis, form a container holding the maximum volume of water.

Examples:

- **Input:** `[1, 8, 6, 2, 5, 4, 8, 3, 7]` $\rightarrow$ **Output:** 49
  Explanation: The walls at indices 1 and 8 have heights 8 and 7, width is 7. Area = min(8,7) * 7 = 49.

Problem: Unique Elements In-Place

Scenario: Given a sorted array, remove duplicate elements in-place so that each unique element appears only once. Return the number of unique elements.

Examples:

- **Input:** `[1, 1, 2]` $\rightarrow$ **Output:** `2 (modified array: [1, 2, _])`

Pattern 1: Two Pointers Pattern – Common Triggers

- The array is **Sorted**.
- You need to find a pair, a triplet, or a subarray.
- You need to "filter" or "rearrange" an array in-place.

Pattern 2: Sliding Window

What is the Sliding Window Pattern?

- **The Core Idea:** Maintain a contiguous window over a linear data structure (array/string) that dynamically expands or contracts.
- **Why it works:** Converts nested $O(n^2)$ loops searching subarrays into a single $O(n)$ pass by reusing computed information.

Dynamic vs. Fixed Window

- **Fixed Window:** Size K is constant. Move window by adding next element and removing the leftmost.
- **Dynamic Window:** Size changes.
  - Expand the window by moving ‘right‘ pointer.
  - If constraint is violated, contract window by moving ‘left‘ pointer until constraint is satisfied again.

Problem: Longest Unique Substring

Scenario: Given a string, find the length of the longest contiguous substring that contains only unique characters.

Examples:

- **Input:** `'abcabcbb'` → **Output:** `3 ('abc')`
- **Input:** `'bbbbb'` → **Output:** `1 ('b')`

Problem: Anagram Substring Search

Scenario: Given two strings s1 and s2, determine if s2 contains a substring that is a permutation (anagram) of s1.

Examples:

- **Input:** `text1 = 'ab', text2 = 'eidbaooo'` → **Output:** `true`
- **Input:** `text1 = 'ab', text2 = 'eidboaoo'` → **Output:** `false`

Problem: Maximum Segment with Two Types

Scenario: Given an array of items, find the maximum length of a contiguous subarray that contains at most two distinct types of items.

Examples:

- **Input:** `[1, 2, 1]` → **Output:** `3`
- **Input:** `[0, 1, 2, 2]` → **Output:** `3 ('[1, 2, 2]')`

Problem: Smallest Window Containing Subsequence

Scenario: Given strings S and T, find the minimum length window substring of S which contains all characters of T (including duplicates). Return empty if no such window exists.

Examples:

- **Input:** `text = 'ADOBECODEBANC', target = 'ABC'` $\rightarrow$ **Output:** `'BANC'`

Pattern 2: Sliding Window – Common Triggers

- Subarray or substring problems.
- Finding shortest, longest, or target length matching constraint.
- Tracking unique elements/frequencies in a range.

Pattern 3: Monotonic Stack

What is a Monotonic Stack?

- **The Core Idea:** A stack that maintains elements in a strictly increasing or decreasing order.
- **How it works:** When a new element is processed, we pop all elements from the stack that violate the monotonic property before pushing the new element.

Monotonic Stack: Types

- **Monotonic Increasing Stack:** Stack elements are sorted in ascending order from bottom to top (e.g. ‘[1, 3, 5, 8]‘). Useful to find the first smaller element to the left/right.
- **Monotonic Decreasing Stack:** Stack elements are sorted in descending order from bottom to top (e.g. ‘[9, 7, 4, 2]‘). Useful to find the first larger element to the left/right.

Problem: Days to Warmer Temperature

Scenario: Given an array of daily temperatures, calculate the number of days you must wait until a warmer day occurs. If no warmer day is possible, output 0.

Examples:

- **Input:** `[73, 74, 75, 71, 69, 72, 76, 73]` $\rightarrow$ **Output:** `[1, 1, 4, 2, 1, 1, 0, 0]`

Problem: Next Greater Element Search

Scenario: Given two arrays nums1 and nums2 where nums1 is a subset of nums2, find the first element in nums2 to the right of each nums1 element that is strictly greater.

Examples:

- **Input:** `array1 = [4,1,2], array2 = [1,3,4,2]` $\rightarrow$ **Output:** `[-1, 3, -1]`

Problem: Volume of Trapped Water

Scenario: Given an elevation map of bar heights where the width of each bar is 1, calculate the total volume of water trapped between the bars after a heavy rain.

Examples:

- **Input:** `[0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]` $\rightarrow$ **Output:** `6`

Problem: Largest Rectangular Area in Histogram

Scenario: Given an array of integers representing heights of consecutive bars in a histogram, find the area of the largest rectangle that can fit inside the histogram boundary.

Examples:

- **Input:** `[2, 1, 5, 6, 2, 3]` $\rightarrow$ **Output:** `10`

Pattern 3: Monotonic Stack – Common Triggers

- Finding the "next greater" or "previous smaller" element in an array.
- Computing areas bounded by heights (e.g. histograms, trapping water).
- Resolving queries in $O(N)$ total time where each index is pushed/popped at most once.

Pattern 4: Prefix Sum & Cumulative Patterns

What is the Prefix Sum Pattern?

- **The Core Idea:** Precompute cumulative sums of an array: ‘prefix[i] = nums[0] + ... + nums[i]‘.
- **Why it works:** Allows you to calculate the sum of any subarray ‘nums[i...j]‘ in $O(1)$ time:

$$\text{Sum}(i, j) = \text{prefix}[j] - \text{prefix}[i - 1]$$

Combining Prefix Sum with HashMap

To find subarrays summing to K in $O(N)$ time:

- Maintain running cumulative sum ‘currentSum‘.
- At each step, we look for a previous prefix sum that equals ‘currentSum - K‘.
- Use a HashMap to store frequencies of prefix sums seen so far.
- **Key Modulo Trick:** For divisibility by K, store ‘currentSum

Problem: Subarrays Summing to Target

Scenario: Given an array of integers and a target value k, return the total count of contiguous subarrays whose elements sum up exactly to k.

Examples:

- **Input:** `array = [1, 1, 1], target = 2` $\rightarrow$ **Output:** `2`

Problem: Subarrays Sum Divisible by K

Scenario: Given an array of integers and a value k, return the number of non-empty contiguous subarrays that have a sum divisible by k.

Examples:

- **Input:** `array = [4, 5, 0, -2, -3, 1], divisor = 5` → **Output:** `7`

Pattern 4: Prefix Sum & Cumulative Patterns – Common Triggers

- Range sum query problems with static arrays.
- Finding subarrays that sum to a target value K.
- Subarrays with properties related to divisibility or modulo arithmetic.

Pattern 5: Cyclic Sort

What is the Cyclic Sort Pattern?

- **The Core Idea:** When an array contains numbers in a range from 1 to n (or 0 to n), we can sort it in $O(n)$ time and $O(1)$ auxiliary space.
- **How it works:** Iterate through the array. If the current number ‘nums[i]‘ is not at its correct index (i.e. ‘nums[i] != i + 1‘), swap it with the number at its correct index ‘nums[nums[i] - 1]‘. Repeat this until the correct number is at index ‘i‘, then move to the next index.

Cyclic Sort Mechanics

- **Index Rule:** A value ‘val‘ belongs at index ‘val - 1‘ (or index ‘val‘ if range is 0 to n).
- **Loop condition:** We only swap when the target index has a different value:

$$\text{nums}[i] \neq \text{nums}[\text{nums}[i] - 1]$$

- **Complexity analysis:** Although we have a nested loop or repeat swaps at the same index, each swap puts at least one number in its correct position. Thus, at most $N - 1$ swaps are performed, leading to $O(N)$ time.

Problem: Disappeared Numbers in Range

Scenario: Given an array of length n where each element is in the range [1, n], find all integers in that range that do not appear in the array.

Examples:

- **Input:** `[4, 3, 2, 7, 8, 2, 3, 1]` → **Output:** `[5, 6]`

Problem: First Missing Positive Integer

Scenario: Given an unsorted integer array, find the smallest positive integer that is not present in the array. Must run in $O(N)$ time and $O(1)$ auxiliary space.

Examples:

- **Input:** `[1, 2, 0]` $\rightarrow$ **Output:** `3`
- **Input:** `[3, 4, -1, 1]` $\rightarrow$ **Output:** `2`

Problem: Duplicate Elements in Range

Scenario: Given an array of integers of size n where 1 <= nums[i] <= n and some elements appear twice, find all duplicate elements.

Examples:

- **Input:** `[4, 3, 2, 7, 8, 2, 3, 1]` → **Output:** `[2, 3]`

Pattern 5: Cyclic Sort – Common Triggers

- Problems dealing with numbers in a continuous range.
- Finding duplicate or missing numbers in an array.
- Finding the first/smallest missing positive number.

Pattern 6: Merge Intervals

What is the Merge Intervals Pattern?

- **The Core Idea:** Process overlapping intervals by sorting them and merging adjacent ranges.
- **Why it works:** Once sorted by start time, overlapping intervals will be adjacent. We can merge them in a single linear pass.

Merging Logic

Given two sorted intervals A and B, they overlap if:

$$B.\text{start} \leq A.\text{end}$$

If they overlap, we merge them into a single interval C where:

$$C.\text{start} = A.\text{start}$$

$$C.\text{end} = \max(A.\text{end}, B.\text{end})$$

Otherwise, they do not overlap, and we add A to our output list and set B as the new active interval.

Problem: Consolidating Overlapping Intervals

Scenario: Given an array of intervals [start, end], merge all overlapping intervals and return a list of non-overlapping intervals covering the same range.

Examples:

- **Input:** `[[1,3],[2,6],[8,10],[15,18]]` → **Output:** `[[1,6],[8,10],[15,18]]`

Problem: Inserting into Non-Overlapping Intervals

Scenario: Given a set of non-overlapping intervals sorted by start time, insert a new interval and merge if it overlaps with existing intervals.

Examples:

- **Input:** `ranges = [[1,3],[6,9]], newRange = [2,5]` → **Output:** `[[1,5],[6,9]]`

Problem: Common Available Time Slots

Scenario: Given schedules of multiple users containing busy intervals, find the free time intervals common to all users.

Examples:

- **Input:** `[[[1,2],[5,6]],[[1,3]],[[4,10]]]` → **Output:** `[[3,4]]`

Pattern 6: Merge Intervals – Common Triggers

- Finding overlapping ranges, free slots, or busy schedules.
- Inserting new intervals into already sorted ranges.
- Finding intersections of multiple lists of intervals.

Pattern 7: Modified Binary Search

What is Modified Binary Search?

- **The Core Idea:** An extension of standard binary search that applies to non-standard sorted structures or search spaces.
- **Why it works:** By dividing the search space in half in $O(1)$ time, we achieve a logarithmic $O(\log N)$ runtime.

Binary Search on Answer Space

Instead of searching indices in an array, we search value ranges (from ‘minPossibleAnswer‘ to ‘maxPossibleAnswer‘):

- Find the midpoint ‘mid‘ of the current range.
- Run a validation function ‘isValid(mid)‘ to check if ‘mid‘ is a possible solution (usually in $O(N)$ time).
- Based on the result:
  - If valid: Try to find a better solution in the lower half (if minimizing).
  - If invalid: Discard the lower half and search the upper half.

Problem: Search in Shifted Sorted Array

Scenario: Search for a target value in a sorted array that has been rotated (shifted circular-wise) at an unknown pivot index. Return its index, or -1 if not found.

Examples:

- **Input:** `array = [4,5,6,7,0,1,2], target = 0` $\rightarrow$
  Output: `4`

Problem: Find Minimum in Shifted Sorted Array

Scenario: Given a sorted array that has been rotated (shifted circular-wise) at an unknown pivot index, find the minimum element in the array.

Examples:

- **Input:** `[3, 4, 5, 1, 2]` $\rightarrow$ **Output:** `1`

Problem: Package Shipping Optimization

Scenario: A conveyor belt carries packages that must be shipped in order within D days. Find the minimum weight capacity of the ship that permits this.

Examples:

- **Input:** `loads = [1,2,3,4,5,6,7,8,9,10], targetDays = 5` $\rightarrow$ **Output:** `15`

Problem: Fast Power Calculation

Scenario: Implement a function to calculate x raised to the power n, i.e., x^n, supporting negative integer powers.

Examples:

- **Input:** `base = 2.00, power = 10` $\rightarrow$ **Output:** `1024.00`
- **Input:** `base = 2.10, power = 3` $\rightarrow$ **Output:** `9.261`

Pattern 7: Modified Binary Search – Common Triggers

- Searching in a rotated sorted array.
- Finding boundaries, peaks, or transition points.
- **Binary Search on Answers:** When checking if a candidate answer is valid takes $O(N)$ time, and the answer is monotonic, we search the answer range.

Pattern 8: Fast & Slow Pointers

What is the Fast & Slow Pointers Pattern?

- **The Core Idea:** Use two pointers traversing a linear data structure (linked list/array) at different speeds.
- **Why it works:** If there is a cycle, the fast pointer (moving 2 steps at a time) will eventually meet the slow pointer (moving 1 step at a time) in $O(N)$ time and $O(1)$ space.

Cycle Detection Mechanics

- **Initialize:** ‘slow = head‘, ‘fast = head‘.
- **Advance:**
  - ‘slow = slow.next‘
  - ‘fast = fast.next.next‘
- **Check:** If ‘slow == fast‘, a cycle is detected! If ‘fast‘ or ‘fast.next‘ is null, there is no cycle.
- **Cycle Start (Floyd’s algorithm):** Once met, reset ‘slow‘ to ‘head‘. Move both pointers 1 step at a time. The point where they meet again is the start of the cycle.

Problem: Cycle Detection in Linked Nodes

Scenario: Determine if a linked node sequence contains a cycle where a node's next pointer points back to a previous node.

Examples:

- **Input:** `node = [3,2,0,-4], cycle at index 1` $\rightarrow$
  Output: `true`

Problem: Center Node of Link Sequence

Scenario: Given a singly linked list, find and return the middle node. If there are two middle nodes, return the second middle node.

Examples:

- **Input:** `[1, 2, 3, 4, 5]` → **Output:** `Node(3)`
- **Input:** `[1, 2, 3, 4, 5, 6]` → **Output:** `Node(4)`

Problem: Find Repeated Number in Constant Space

Scenario: Given an array of size n+1 where each integer is in range [1, n], find the repeated number using only constant extra space and without modifying the array.

Examples:

- **Input:** `[1, 3, 4, 2, 2]` $\rightarrow$ **Output:** `2`

Pattern 8: Fast & Slow Pointers – Common Triggers

- Cycle detection in a linked list.
- Finding the starting node of a cycle.
- Finding the middle node of a list (when the fast pointer reaches the end, the slow pointer is in the middle).
- Identifying duplicates in structures where values can be treated as indices.

Pattern 9: Tree Depth First Search (DFS)

Tree Depth First Search (DFS)

- **The Core Idea:** Explore as deep as possible along each branch of a tree before backtracking.
- **Implementation:** Typically implemented using recursion (which uses the call stack implicitly) or an explicit Stack.

DFS Traversal Types

Depending on when the current node is processed relative to its children:

- **Pre-order (Root, Left, Right):** Processing parent before children. Useful for copying trees or serialization.
- **In-order (Left, Root, Right):** Processing left child, then parent, then right child. Yields sorted keys in a Binary Search Tree (BST).
- **Post-order (Left, Right, Root):** Processing children before parent. Useful for bottom-up computation (e.g., node height, subproblem accumulation).

Problem: Root-to-Leaf Target Sum

Scenario: Given a binary tree and a targetSum, return true if the tree has a root-to-leaf path such that adding up all the values along the path equals targetSum.

Examples:

- **Input:** `tree = [5,4,8,11,null,13,4,7,2,null,null,null,1], target = 22` $\rightarrow$ **Output:** `true`

Problem: Any-Path Sum Count

Scenario: Given a binary tree, find the number of paths that sum to a given target value. The path does not need to start or end at the root or leaf, but must go downwards.

Examples:

- **Input:** `tree = [10,5,-3,3,2,null,11,3,-2,null,1]`, `target = 8` $\rightarrow$ **Output:** `3`

Problem: Validate Binary Search Tree Structure

Scenario: Determine if a given binary tree is a valid Binary Search Tree (BST), where left children are smaller and right children are larger.

Examples:

- **Input:** `[2, 1, 3]` $\rightarrow$ **Output:** `true`
- **Input:** `[5, 1, 4, null, null, 3, 6]` $\rightarrow$ **Output:** `false`

Problem: Lowest Common Ancestor

Scenario: Find the lowest common ancestor node of two given nodes in a binary tree.

Examples:

- **Input:** `tree = [3,5,1,6,2,0,8], target1 = 5, target2 = 1` $\rightarrow$ **Output:** `3`

Problem: Maximum Path Sum in Tree

Scenario: Find the maximum path sum in a binary tree, where a path is any sequence of nodes from some starting node to any node in the tree along parent-child connections.

Examples:

- **Input:** `[-10, 9, 20, null, null, 15, 7]` $\rightarrow$ **Output:** `42`

Pattern 9: Tree Depth First Search (DFS) – Common Triggers

- Finding paths that sum to a target value.
- Computing properties of nodes recursively (height, balance).
- Validating BST properties.
- Exploring combinations/subtrees.

Pattern 10: Tree Breadth First Search (BFS)

Tree Breadth First Search (BFS)

- **The Core Idea:** Traverse a tree level-by-level, processing all nodes at depth d before any node at depth $d + 1$.
- **Implementation:** Typically implemented iteratively using a Queue (FIFO).

Level Size Strategy

To process nodes level by level rather than as a continuous queue stream:

- At the start of each level iteration, count the queue size: ‘val levelSize = queue.size‘.
- Run a nested loop exactly ‘levelSize‘ times.
- Inside the loop:
  - Pop a node from the queue.
  - Enqueue its children (left, then right).
- Any logic scoped to a level (e.g., calculating level average or reversing order) can be done cleanly.

Problem: Level-by-Level Tree Traversal

Scenario: Given a binary tree, return the level order traversal of its nodes' values (from left to right, level by level).

Examples:

- **Input:** `[3, 9, 20, null, null, 15, 7]` $\rightarrow$ **Output:** `[[3], [9, 20], [15, 7]]`

Problem: Zigzag Level Order Traversal

Scenario: Given a binary tree, return the zigzag level order traversal of its nodes' values (left-to-right, then right-to-left for the next level, and alternate).

Examples:

- **Input:** `[3, 9, 20, null, null, 15, 7]` $\rightarrow$ **Output:** `[[3], [20, 9], [15, 7]]`

Problem: Connecting Right Neighbors in Tree

Scenario: Given a perfect binary tree, populate each next pointer to point to its next right node. If there is no next right node, set it to NULL.

Examples:

- **Input:** `[1, 2, 3, 4, 5, 6, 7]` $\rightarrow$ **Output:** `[1, #, 2, 3, #, 4, 5, 6, 7, #]`

Problem: Word Transformation Sequence

Scenario: Given two words (beginWord, endWord) and a dictionary, find the length of the shortest transformation sequence from beginWord to endWord, changing one letter at a time.

Examples:

- **Input:** `startWord = 'hit', endWord = 'cog', wordList = ['hot','dot','dog','lot','log','cog']`
  $\rightarrow$ **Output:** `5`

Pattern 10: Tree Breadth First Search (BFS) – Common Triggers

- Level order traversals or printing.
- Finding the minimum depth of a tree.
- Tracking "level" boundaries (e.g. connecting next pointers).
- Finding the shortest path in unweighted graphs or state transitions.

Pattern 11: Matrix Traversal (The "Islands" Pattern)

What is the Matrix Traversal Pattern?

- **The Core Idea:** Treat a 2D grid (matrix) as an implicit graph where each cell (r, c) is a node, and its adjacent neighbors (up, down, left, right) are connected by edges.
- **Traversal:** Use Depth First Search (DFS) or Breadth First Search (BFS) to explore connected regions.

Matrix Traversal Mechanics

- **Boundary Check:** Always ensure row ‘r‘ and column ‘c‘ are valid:
$$0 \leq r < R \quad \text{and} \quad 0 \leq c < C$$
- **Directions Array:** Use a utility array to iterate over neighbors cleanly:
$$\text{dirs} = [(-1, 0), (1, 0), (0, -1), (0, 1)]$$
- **Visited Tracking:** Mark grid cells as visited (e.g., set ‘’1‘‘ to ‘’0‘‘ in-place if allowed, or use a ‘visited‘ boolean matrix) to prevent infinite loops.

Problem: Counting Isolated Grid Clusters

Scenario: Given a 2D grid of '1's (land) and '0's (water), count the number of isolated land clusters (islands) surrounded by water cardinally.

Examples:

- **Input:** `matrix = [['1','1','0'],['1','1','0'],['0','0','0']]` $\rightarrow$ **Output:** `1`

Problem: Largest Connected Grid Cluster

Scenario: Find the maximum area of a connected grid cluster (island) in a 2D grid, where area is the number of cardinally connected land cells.

Examples:

- **Input:** `matrix = [[0, 1], [1, 1]]` $\rightarrow$ **Output:** `3`

Problem: Spreading Contagion in Grid

Scenario: In a 2D grid containing fresh items, contaminated items, and empty spaces, find the minimum time elapsed until all fresh items are contaminated. Spreads cardinally. Return -1 if impossible.

Examples:

- **Input:** `matrix = [[2,1,1],[1,1,0],[0,1,1]]` $\rightarrow$ **Output:** `4`

Problem: Water Flow Boundaries

Scenario: Given a 2D matrix of heights, find all coordinates from which water can flow cardinally to both the left/top ocean (Pacific) and the right/bottom ocean (Atlantic).

Examples:

- **Input:** `matrix = [[1,2],[2,1]]` $\rightarrow$ **Output:** `[[0,0],[0,1],[1,0],[1,1]]`

Pattern 11: Matrix Traversal (The "Islands" Pattern) – Common Triggers

- Counting distinct blocks (e.g. islands, provinces).
- Simulating cell expansion over time (e.g. rotting oranges, fire spreading).
- Finding paths or reachable regions from boundaries.

Pattern 12: Union-Find (Disjoint Set)

What is the Union-Find Pattern?

- **The Core Idea:** A data structure that tracks partition of a set into disjoint subsets.
- **Operations:**
  - **Find:** Identify which subset an element belongs to (returns root parent).
  - **Union:** Join two disjoint subsets into a single subset.

Optimizations: Path Compression & Union by Rank

Without optimization, a tree can become a line ($O(N)$ lookup).

- **Path Compression:** During ‘find(x)‘, make every node in the search path point directly to the root root. This flattens the tree structure:

$$\text{parent}[x] = \text{find}(\text{parent}[x])$$

- **Union by Rank/Size:** Always attach the smaller tree under the root of the larger tree.
- **Complexity:** Combined, they achieve nearly constant time operations: $O(\alpha(N))$ where α is the Inverse Ackermann function.

Problem: Connected Components in Network

Scenario: Given an adjacency matrix representing relationships (e.g. friendship), find the total number of connected groups/components in the network.

Examples:

- **Input:** `adjacencyMatrix = [[1,1,0],[1,1,0],[0,0,1]]`
  $\rightarrow$ **Output:** `2`

Problem: Locating Cycle-Creating Edge

Scenario: Given an undirected graph with n nodes and n edges containing exactly one cycle, find the edge that can be removed to turn the graph into a tree.

Examples:

- **Input:** `[[1,2], [1,3], [2,3]]` → **Output:** `[2,3]`

Problem: Merging Overlapping Identifiers

Scenario: Given a list of accounts with names and emails, merge accounts that share common emails, returning a sorted list of merged emails.

Examples:

- **Input:** `[['John','a@m.com','b@m.com'], ['John','c@m.com','a@m.com']]` $\rightarrow$ **Output:** `[['John','a@m.com','b@m.com','c@m.com']]`

Pattern 12: Union-Find (Disjoint Set) – Common Triggers

- Dynamic connectivity (are nodes X and Y in the same group?).
- Detecting cycles in undirected graphs.
- Merging accounts, groups, or equivalence sets.

Pattern 13: Top ‘K’ Elements (Heap / Priority Queue)

What is the Top 'K' Elements Pattern?

- **The Core Idea:** Find the K largest, smallest, or most frequent elements in a large collection.
- **Why it works:** Instead of sorting the entire array in $O(N \log N)$ time, we can maintain a Heap/Priority Queue of size K.
- **Heaps selection rule:**
  - To find the **K largest** elements: Use a **Min-Heap**. When size exceeds K, pop the smallest element. The heap will retain the K largest values.
  - To find the **K smallest** elements: Use a **Max-Heap**. When size exceeds K, pop the largest.
- **Complexity:** $O(N \log K)$ time and $O(K)$ extra space.

Problem: K Most Frequent Elements

Scenario: Given an integer array and an integer k, return the k most frequent elements in the array. You may return the answer in any order.

Examples:

- **Input:** `array = [1,1,1,2,2,3], limit = 2` → **Output:** `[1, 2]`

Problem: K Nearest Coordinates to Origin

Scenario: Given an array of points on the X-Y plane and an integer k, return the k closest points to the origin (0, 0) using Euclidean distance.

Examples:

- **Input:** `coordinates = [[1,3],[-2,2]], limit = 1` $\rightarrow$ **Output:** `[[-2,2]]`

Problem: CPU Task Scheduler

Scenario: Given a list of CPU tasks represented by characters and a cooldown period n, find the minimum CPU intervals required to complete all tasks.

Examples:

- **Input:** `jobs = ['A','A','A','B','B','B'], cooldown = 2` $\rightarrow$ **Output:** `8`

Problem: Rearrange No-Adjacent-Duplicate Characters

Scenario: Given a string, rearrange its characters so that any two adjacent characters are not the same. Return empty string if impossible.

Examples:

- **Input:** `'aab'` $\rightarrow$ **Output:** `'aba'`
- **Input:** `'aaab'` $\rightarrow$ **Output:** `''`

Pattern 14: Dynamic Programming

What is Dynamic Programming (DP)?

- **The Core Idea:** Solve complex problems by breaking them down into simpler, overlapping subproblems and caching their results to avoid redundant calculations.
- **Key Properties:**
  - **Overlapping Subproblems:** Solving the same subproblems repeatedly.
  - **Optimal Substructure:** The optimal solution of the problem can be constructed from optimal solutions of its subproblems.
- **Approaches:**
  - **Top-Down (Memoization):** Recursive, start from target, solve subproblems and save values in a cache.
  - **Bottom-Up (Tabulation):** Iterative, start from base cases, fill a table (array/matrix) up to target.

Dynamic Programming Sub-patterns

Most interview DP problems fall into standard subclasses:

- **Fibonacci Numbers:** ‘dp[i] = dp[i-1] + dp[i-2]‘ (Climbing Stairs).
- **0/1 Knapsack:** Items can be chosen at most once. Decide to include or exclude an item: ‘dp[i][w] = max(exclude, include)‘.
- **Unbounded Knapsack / Coin Change:** Items can be chosen multiple times.
- **Longest Common Subsequence (LCS) / Edit Distance:** String matching.
- **Longest Increasing Subsequence (LIS):** Decisions based on previous elements.

Problem: Distinct Ways to Climb Stairs

Scenario: You are climbing a staircase that takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

- **Input:** 2 $\rightarrow$ **Output:** 2
- **Input:** 3 $\rightarrow$ **Output:** 3

Concept: 0/1 Knapsack

Problem: Minimum Coins for Change

Scenario: Given coins of different denominations and a total amount, write a function to compute the fewest number of coins that you need to make up that amount.

Examples:

- **Input:** `denominations = [1,2,5], targetAmount = 11`
  $\rightarrow$ **Output:** `3`

Problem: Combinations count for Change

Scenario: Given coins of different denominations and a total amount, return the number of combinations that make up that amount.

Examples:

- **Input:** `denominations = [1,2,5], targetAmount = 5` → **Output:** `4`

Problem: Longest Strictly Increasing Subsequence

Scenario: Given an integer array nums, return the length of the longest strictly increasing subsequence.

Examples:

- **Input:** `[10,9,2,5,3,7,101,18]` $\rightarrow$ **Output:** `4`

Problem: String Segmentation using Dictionary

Scenario: Given a string and a dictionary of words, return true if the string can be segmented into a space-separated sequence of dictionary words.

Examples:

- **Input:** `text = 'leetcode', dictionary = ['leet','code']` $\rightarrow$ **Output:** `true`

Pattern 15: Backtracking & DFS

What is the Backtracking Pattern?

- **The Core Idea:** A systematic search algorithm that builds candidate solutions incrementally, and abandons a candidate ("backtracks") as soon as it determines the candidate cannot lead to a valid solution.
- **How it works (The Three Steps):**
  1. **Choose:** Add an element to the current path.
  2. **Explore:** Recursively call the search function with the new state.
  3. **Unchoose (Backtrack):** Remove the element from the current path to return to the previous state.

Problem: Well-Formed Parentheses Combinations

Scenario: Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Examples:

- **Input:** 3 $\rightarrow$ **Output:**
  `['((()))','(()())','(())()','()(())','()()()']`

Problem: Combination Sum to Target

Scenario: Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target.

Examples:

- **Input:** `values = [2,3,6,7], target = 7` $\rightarrow$ **Output:** `[[2,2,3],[7]]`

Problem: Palindrome Substring Partitions

Scenario: Given a string, partition s such that every substring of the partition is a palindrome. Return all possible palindrome partitioning of s.

Examples:

- **Input:** `'aab'` $\rightarrow$ **Output:** `[['a','a','b'],['aa','b']]`

Problem: Word Grid Search

Scenario: Given an m x n grid of characters and a string word, return true if word exists in the grid. The word can be constructed from letters of sequentially adjacent cells.

Examples:

- **Input:** `matrix = [['A','B','C','E'],['S','F','C','S']], word = 'ABCCED'` → **Output:** `true`

Problem: N-Queens Placement

Scenario: Place n queens on an n x n chessboard such that no two queens attack each other. Return all distinct solutions.

Examples:

- **Input:** $4 \rightarrow$ **Output:** `[[ '.Q..', '...Q', 'Q...', '..Q.' ], [ '..Q.', 'Q...', '...Q', '.Q..' ]]`

Pattern 15: Backtracking & DFS – Common Triggers

- Generating all permutations, combinations, or subsets.
- Solving constraint satisfaction puzzles (e.g. N-Queens, Sudoku).
- Exploring word combinations on a grid.

Pattern 16: Greedy Algorithms

What are Greedy Algorithms?

- **The Core Idea:** An algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate local benefit ("greedy" choice).
- **Why it works:** For certain problems, making locally optimal decisions leads to a globally optimal solution.
- **Tradeoffs:**
  - Highly efficient (usually $O(N)$ or $O(N \log N)$).
  - Difficult to mathematically prove correctness.
  - If greedy fails, you must fall back to Dynamic Programming (which checks all combinations).

Problem: Circular Route Fuel Stations

Scenario: Given gas and cost arrays for circular route fuel stations, find the starting station index from which you can travel around the circuit once without running out of gas.

Examples:

- **Input:** `fuel = [1,2,3,4,5], cost = [3,4,5,1,2]` $\rightarrow$ **Output:** `3`

Problem: Min Candy Distribution

Scenario: There are n children standing in a line. Each child has a rating. Distribute candies such that each child has at least 1, and children with higher ratings than neighbors get more.

Examples:

- **Input:** `[1,0,2]` $\rightarrow$ **Output:** `5`

Problem: Line Reconstruction by Height Attributes

Scenario: Reconstruct a queue where each person is represented by [h, k] where h is their height and k is the number of people in front of them who are taller or equal.

Examples:

- **Input:** `[[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]` $\rightarrow$ **Output:** `[[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]`

Pattern 16: Greedy Algorithms – Common Triggers

- Scheduling, interval selection, minimizations.
- Gas stations, container loading, coin distributions (canonical cases).

Pattern 17: Data Streams & Online Algorithms

Data Streams & Online Algorithms

- **The Core Idea:** Process data that arrives continuously over time, rather than having the entire dataset available in memory upfront.
- **Constraint:** Algorithms must run in real-time with limited space ($O(1)$ or $O(K)$ where K is the query window size) and time complexity per element.
- **Key Techniques:**
  - **Two Heaps:** Balancing a Max-Heap and Min-Heap to find median in $O(1)$ time.
  - **Reservoir Sampling:** Selecting a random sample from a stream of unknown size.
  - **Double Linked List + HashMap:** Designing fast cache policies (like LRU/LFU cache) with $O(1)$ insertion, lookup, and deletion.

Problem: Running Median from Data Stream

Scenario: Design a data structure that supports adding numbers from a data stream and finding the median of the numbers added so far.

Examples:

- **Input:** `addNum(1), addNum(2), findMedian()` $\rightarrow$
  Output: `1.5`

Problem: Random Pick Index from Duplicates

Scenario: Given an integer array with duplicate elements, randomly pick an index of a given target number with equal probability.

Examples:

- **Input:** `array = [1,2,3,3,3], pick(3)` $\rightarrow$ **Output:** `2, 3, or 4`

Problem: Moving Average in Sliding Window

Scenario: Given a stream of integers and a window size, calculate the moving average of all integers in the sliding window.

Examples:

- **Input:** `capacity = 3, next(1), next(10), next(3), next(5)` $\rightarrow$ **Output:** `1.0, 5.5, 4.66, 6.0`

Problem: LRU Cache Design

Scenario: Design a data structure that follows the constraints of a Least Recently Used (LRU) cache with get and put operations in O(1) time.

Examples:

- **Input:** `capacity = 2, put(1,1), put(2,2), get(1), put(3,3)` $\rightarrow$ **Output:** `get(1)->1, get(2)->-1`

Pattern 18: Advanced Patterns

Advanced Algorithmic Patterns

For Senior and Staff level roles, interviewers often present advanced structures:

- **Segment Tree:** A binary tree used for storing intervals or segments. It allows querying a range (sum, min, max) and modifying elements in $O(\log N)$ time.
- **Topological Sort:** Ordering vertices in a directed acyclic graph (DAG) such that for every directed edge uv, vertex u comes before v. (Kahn's BFS or DFS).
- **Sweep Line:** A geometry paradigm that solves range/interval problems by sorting boundary events (start/end) and sweeping a imaginary line across them.
- **Trie (Prefix Tree):** A search tree used to store associative keys (typically strings) efficiently. Excellent for autocompletion, spelling checkers, prefix matching.

Problem: Dynamic Array Range Sum Query

Scenario: Given an array, implement a structure to calculate range sum queries [left, right] and update elements at index in logarithmic time.

Examples:

- **Input:** `array = [1,3,5], sumRange(0,2), update(1,2), sumRange(0,2)` → **Output:** `9, then 8`

Problem: Course Dependency Resolution

Scenario: Determine if you can finish all courses given the total number of courses and prerequisite dependencies.

Examples:

- **Input:** `coursesCount = 2, dependencies = [[1,0]]` $\rightarrow$ **Output:** `true`
- **Input:** `coursesCount = 2, dependencies = [[1,0],[0,1]]` $\rightarrow$ **Output:** `false`

Problem: Outline of Bounded Buildings

Scenario: Given building locations and heights, return the key points that outline the collective skyline contour.

Examples:

- **Input:** `[[2,9,10],[3,7,15]]` $\rightarrow$ **Output:** `[[2,10],[3,15],[7,0]]`

Problem: Prefix Tree Implementation

Scenario: Implement a trie (prefix tree) with insert, search, and startsWith methods.

Examples:

- **Input:** `insert('apple'), search('apple'), startsWith('app')` → **Output:** `true, true`

Pattern 19: Miscellaneous Techniques

Miscellaneous Interview Techniques

Some interview questions don't fit into single patterns, but require a combination of general skills:

- **In-place Grid Simulation:** Modifying a matrix without using extra memory by using rows/columns headers as state markers (e.g. Set Matrix Zeroes).
- **Math & Coordinate Transformations:** Mirroring, swapping, or transposing grids (e.g. Rotate Image).
- **Stack Evaluation:** Parsing arithmetic strings or expressions sequentially (e.g. Evaluate Reverse Polish Notation).
- **Bitmasking or Row/Column hashing:** Validating board boundaries quickly (e.g. Valid Sudoku).

Problem: Spread Zeroes in 2D Array

Scenario: Given an m x n integer matrix, if an element is 0, set its entire row and column to 0 in-place.

Examples:

- **Input:** `[[1,1,1],[1,0,1],[1,1,1]]` → **Output:** `[[1,0,1],[0,0,0],[1,0,1]]`

Problem: Validate Sudoku Board State

Scenario: Determine if a 9 x 9 Sudoku board configuration is valid, checking rows, columns, and 3x3 sub-grids.

Examples:

- **Input:** `grid = ...` $\rightarrow$ **Output:** `true/false`

Problem: Rotate 2D Matrix In-Place

Scenario: Given an n x n 2D matrix representing an image, rotate the image by 90 degrees clockwise in-place.

Examples:

- **Input:** `[[1,2,3],[4,5,6],[7,8,9]]` → **Output:** `[[7,4,1],[8,5,2],[9,6,3]]`

Problem: Postfix Expression Evaluation

Scenario: Evaluate the value of an arithmetic expression in Reverse Polish Notation (RPN), supporting standard operators.

Examples:

- **Input:** `['2','1','+','3','*']` $\rightarrow$ **Output:** `9`

Section 22 — Summary & The Modern Interview Landscape

Systematic Problem-Solving Template

How to tackle any unknown problem in an interview:

1. **Clarify:** Ask questions about constraints (n, value limits, sorted status, memory limits).
2. **Example:** Dry run a custom, non-trivial test case.
3. **Brute Force:** State the obvious solution ($O(N^2)$ or $O(2^N)$), explaining its bottleneck.
4. **Optimize:** Apply pattern matching (e.g. "Do I need a window?", "Can a map help?").
5. **Code:** Write clean, modular code, stating edge cases.
6. **Dry Run:** Trace the code line-by-line with the example.

The Engineering Manager's Lens

Interviews are a signal of how you will perform as a team member:

- **Tradeoffs:** Be vocal about time vs. space complexity tradeoffs.
- **Production-Ready Code:** Favor clean modular helper methods, input validation, and proper typing over dense one-liners.
- **Collaboration:** Treat the interviewer as a teammate. Don't go silent for minutes; explain your thought flow continuously.

Redefining the SDLC in the AI Era

How AI coding agents (like Gemini, Copilot) change the baseline expectation:

- **Syntax is Cheap:** Being a human compiler or memorizing syntax is no longer the primary value.
- **Logic & System Design is King:** Focus on high-level reasoning, system architecture, identifying constraints, and verifying correctness.
- **Coding speed vs. System security:** Writing code quickly is easy; ensuring it is robust, secure, and fits into the existing framework is the real challenge.

Recap & Roadmap Beyond the Course

- You have studied 20+ crucial algorithmic patterns!
- **Next Steps:**
  - Practice spaced repetition (solve a problem, review it after 3 days, then 7 days).
  - Conduct mock whiteboard interviews with peers.
  - Build intuition over syntax.

Congratulations on completing the course!